厦门大学 2016 年期中考试

# UNIX 系统编程

**一、简答题**

（1）如果打开文件时使用了 O_APPEND 标志，那么能否在该文件的任意位置上读写？如果不使用这个标志，能否完成相同功能？可能存在什么问题？

（2）在读写磁盘文件时，每次使用系统调用 read 和 write 是否都意味着读写磁盘驱动器？为什么？

（3）如果一个文件，他的权限被设置为文件的所有者不可读，那么文件的所有者是否可将其设置为自己可读？如果其他用户可读，那么文件的所有者是否能将文件改为自己可读？

（4）硬链接和符号链接有什么区别？

（5）什么是捕获信号？所有信号都是可以捕获的吗？

**二、用户 ID（uid），实际用户 ID（ruid），有效用户 ID（euid），设置-用户-ID（suid）和保存的设置-用户-ID（ssuid）是如何产生的？各自的意义是什么？**

**三、阅读教材程序清单 4-7（P99），回答问题：**

（1）dopath 函数返回值的意义是什么？

（2）程序遍历目录树的次序是深度优先还是广度优先？为什么？

（3）程序遍历目录树时，在每个节点执行的函数是什么？完成什么功能？

（4）在 dopath 函数中，为什么需要语句（不含引号）“ptr[-1]=0;”？

（5）如果在程序中使用系统调用 stat 获取目录项信息会有什么不同的结果？

**四、下面代码是利用 sigaction 函数实现 signal 函数的可靠信号版本。若使用这个 signal 函数，则**

```
#include <signal.h>
#include "ourhdr.h"

Sigfunc *
signal(int signo, Sigfunc *func) {
    struct sigaction act, oact;

    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;

    if (signo == SIGALRM) {
#ifdef SA_INTERRUPT
        act.sa_flags |= SA_INTERRUPT; /* SunOS */
#endif
    } else {
#ifdef SA_RESTART
        act.sa_flags |= SA_RESTART; /* SVR4, 4.4BSD */
#endif
    }
```

```
    if (sigaction(signo, &act, &oact) < 0)
        return (SIG_ERR);
    return (oact.sa_handler);
}
```

（1）进入信号处理函数时，是否自动阻塞相同的信号?

（2）进入信号处理函数后，缺省的信号处理方式是否自动恢复?

（3）信号是否排队?

（4）使用此 signal 函数的进程，若其子进程终止，则系统是否会向它发送信号?

（5）若一个阻塞的低速系统调用被信号中断，那么哪些信号不能使这个系统调用重启?

**五、下面的程序是函数 abort 的 POSIX.1 实现。请说明，**

```
#include <stdlib.h>
#include <unistd.h>

void abort(void) /* POSIX-style abort() function */
{
    sigset_t mask;
    struct sigaction action;

    /* caller can't ignore SIGABRT, if so reset to default */
    sigaction(SIGABRT, NULL, &action);
    if (action.sa_handler == SIG_IGN) {
        action.sa_handler = SIG_DFL;
        sigaction(SIGABRT, &action, NULL);
    }

    if (action.sa_handler == SIG_DFL) {
        fflush(NULL); /* flush all open stdio streams */
        /* caller can't block SIGABRT: make sure it's unblocked */
        sigfillset(&mask);
        sigdelset(&mask, SIGABRT); /* mask has only SIGABRT turned off */
        sigprocmask(SIG_SETMASK, &mask, NULL);

        kill(getpid(), SIGABRT); /* send the signal */
    }

    /* if we're here, process caught SIGABRT and returned */
    fflush(NULL); /* flush all open stdio streams */

    action.sa_handler = SIG_DFL;
    sigaction(SIGABRT, &action, NULL); /* reset disposition to default */
    sigprocmask(SIG_SETMASK, &mask, NULL); /* just in case... */
    kill(getpid(), SIGABRT); /* and one more time */
    exit(1); /* this should never be executed... */
}
```

1、第一个 if 语句的作用是什么?

2、为什么要调用两次 fflush(NULL)?

3、为什么要两次调用 kill?

4、POSIX.1 要求 abort 在终止调用进程前，应当对所有打开的标准 I/O 流调用 fclose。这个实现是否做到了？为什么?